

SonicProtoPNet: un modello interpretabile per la classificazione audio

Dario Mezzasalma

Gennaio 2026

1 Introduzione

In questo elaborato ho implementato un modello denominato SonicProtoPNet da utilizzare su un task di classificazione multi-classe. SonicProtoPNet è un modello interpretabile basato sull'apprendimento di prototipi utili ai fini della classificazione e della spiegazione. Questo modello è costituito da tre blocchi distinti:

1. **Blocco convoluzionale** f rappresentato da una backbone convoluzionale pre-addestrata.
2. **Prototype Layer** g costituito da un insieme di prototipi appresi durante il training.
3. **Layer lineare** h .

Il *prototype layer* rappresenta il punto cruciale per la spiegabilità di una certa decisione presa dal modello. Questo layer è costituito da un insieme di m prototipi ognuno dei quali calcola la distanza L^2 tra se stesso e tutte le patch dell'output convoluzionale per produrre mappe di attivazione.

Gli input alla rete sono segnali audio che, per questioni di compatibilità con il blocco convoluzionale, seguono uno step di pre-processing dove vengono trasformati in spettrogrammi (immagini ad un singolo canale).

I blocchi costituenti la rete seguono un processo di addestramento non simultaneo che è caratterizzato da tre step principali:

- **Joint training:** si aggiornano i pesi dell'intera rete eccetto quelli dell'ultimo layer lineare.
- **Prototype projection.**
- **Last Layer Convex Optimization:** si aggiornano solamente i pesi dell'ultimo layer lineare.

2 Preprocessing dei dati

Il dataset con cui ho lavorato è *Nsynth* contenente 305.979 tracce audio della durata di 4 secondi divise su 3 split: *training*, *validation* e *test*. Ogni esempio nel dataset Nsynth rappresenta una nota musicale. Ciascuna nota è registrata utilizzando strumenti diversi ed è annotata con informazioni relative allo strumento che la produce. In questo lavoro ho scelto come etichetta il campo *instrument family*, che assume 11 valori distinti (come in figura 1) corrispondenti alle principali macro-famiglie di strumenti. In alternativa, utilizzando il campo *instrument*, il problema di classificazione avrebbe coinvolto 1006 classi differenti.

Index	ID
0	bass
1	brass
2	flute
3	guitar
4	keyboard
5	mallet
6	organ
7	reed
8	string
9	synth_lead
10	vocal

Figure 1: Classi di Nsynth

Facendo un'analisi esplorativa ho osservato che il dataset risulta sbilanciato (come in figura 2) ed esempi della classe *synth lead* appaiono esclusivamente nel training set. Per quest'ultimo motivo ho deciso di scartare gli esempi di tale classe ottenendo così un problema di classificazione su 10 classi.

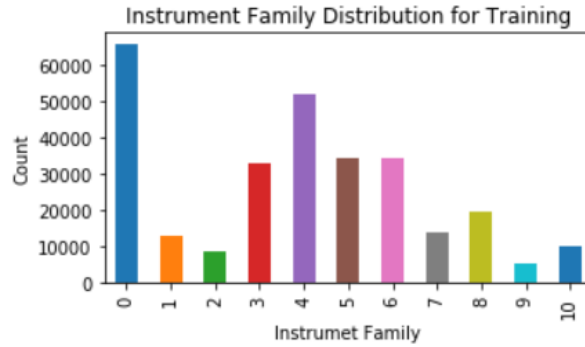


Figure 2: Distribuzione delle classi relative al campo *instrument family*

Queste tracce audio necessitano di essere convertite in spettrogrammi conformi alle specifiche architetturali della rete. Per la generazione degli spettrogrammi ho scelto i seguenti valori per i parametri:

- **sample rate:** 16000
- **n fft:** 2048
- **hop length:** 487
- **n mels:** 128

Questa scelta mi ha permesso di ottenere degli spettrogrammi-Mel ad un singolo canale di risoluzione 128x128 (come in figura 3).

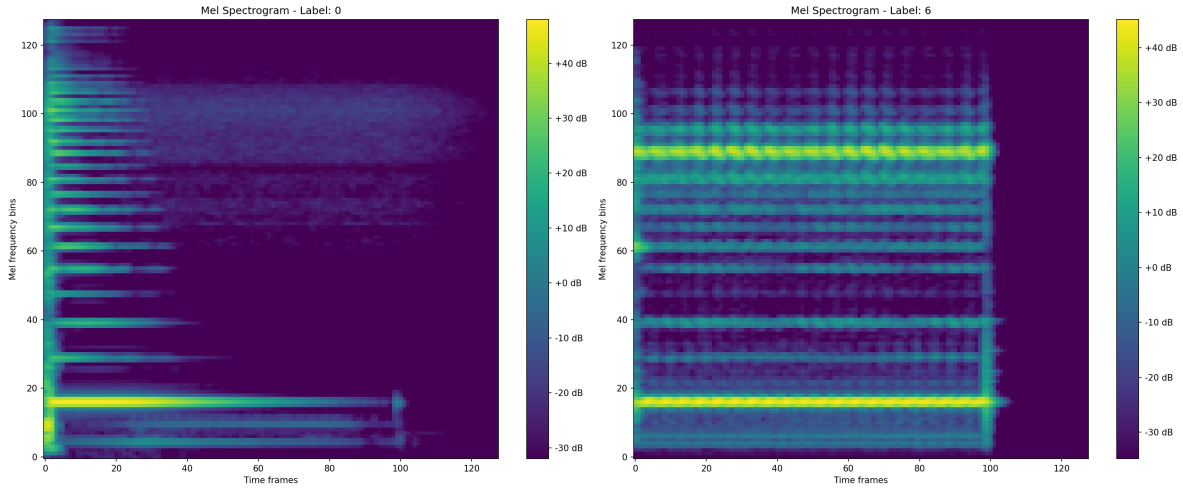


Figure 3: Esempi casuali di spettrogrammi-Mel. Sinistra: esempio di classe *bass*. Destra: esempio di classe *organ*.

3 Esperimenti

Per lo svolgimento degli esperimenti è stato utilizzato un ulteriore set di dati, che identificherò come *artificiale*, (la cui cardinalità è molto vicina a quella del test set) generato in laboratorio e costruito sulle stesse classi del dataset NSynth. Gli obiettivi degli esperimenti sono principalmente i seguenti:

- Valutazione delle *performance* su entrambi i set di dati.
- Analisi del comportamento dei *prototipi* in funzione dei due set di dati.

Poiché il processo di training non avviene in modo simultaneo sull'intera rete, sono stati adottati iperparametri degli ottimizzatori differenti a seconda della fase di addestramento:

- **Warm-up epochs:** fase iniziale in cui viene addestrato l'intero modello ad eccezione della backbone, con l'obiettivo di adattare progressivamente i layer superiori. Negli esperimenti $warm - epochs = 3$.
- **Joint epochs:** fase in cui vengono addestrati tutti i componenti del modello, ad esclusione del layer lineare finale.
- **Prototype projection e Last-only:** ogni 5 epoche di training congiunto (*joint*), vengono eseguite in sequenza queste due fasi di ottimizzazione.

Per ciascuna di queste fasi, l'ottimizzatore è *Adam* con i learning rate diversificati a seconda della porzione di rete che si addestra.

```
# Learning rates and schedulers
joint_optimizer_lrs = {'features': 1e-5,
                      'add_on_layers': 3e-3,
                      'prototype_vectors': 3e-3}
joint_lr_step_size = 5

warm_optimizer_lrs = {'add_on_layers': 3e-3,
                     'prototype_vectors': 3e-3}

last_layer_optimizer_lr = 1e-4
```

Figure 4: Iperparametri relativi agli ottimizzatori

Gli esperimenti effettuati sono costruiti sull'idea di rendere più complesso il modello. Posso agire principalmente su tre componenti tra cui:

- **Backbone:** passando da una backbone più leggera come *VGG 13* ad una più profonda come *VGG 19*.
- **Prototype Layer:** incrementando il numero di prototipi da apprendere.
- **Quantità di dati:** incrementando il quantitativo di esempi visti durante il processo di training.

3.1 Valutazione delle performance

Training completed in 97.42289073467255 minutes	Training completed in 157.5843182206154 minutes
test	test
accuracy: 70.6298828125%	accuracy: 74.169921875%
test	test
accuracy: 9.297052154195011%	accuracy: 10.052910052910052%

Figure 5: Test accuracy su set reale e artificiale nei due casi.

```

Early stop training because: validation acc converged
Best validation accuracy: 0.7552
Training completed in 211.97912244002023 minutes
test
accuracy: 75.6103515625%
test
accuracy: 8.18846056941295%

```

Figure 6: Test accuracy su set reale e artificiale nel caso 3

Come da figura 5 si può subito osservare come il processo di training sia stato dispendioso in termini di tempo. In tutti i casi il modello è stato allenato fino a raggiungere la convergenza sul validation set.

3.1.1 Caso 1: immagine a sinistra in 5

Il primo caso considera un modello di complessità leggermente inferiore. Il backbone adottato è VGG-13, con 50 prototipi, e un training set composto da circa 50.000 esempi, pari a circa un quinto dell'intero insieme di dati disponibile.

3.1.2 Caso 2: immagine a destra in 5

Il secondo caso, invece, prevede l'utilizzo di una *VGG-19* come backbone, 100 prototipi e un training set composto da circa 95.000 esempi, pari a circa un terzo dell'intero training set.

3.1.3 Caso 3: immagine 6

Il terzo caso è identico al precedente con la sola eccezione che il numero di prototipi è pari a 1000 (100 prototipi per classe).

Le performance sono piuttosto simili sia sul set reale che su quello artificiale in tutte le configurazioni.

Analizziamo adesso la matrice di confusione per comprendere meglio su quali classi il modello sbaglia più frequentemente.

La classe su cui il modello commette il maggior numero di errori è la classe 9, corrispondente alla categoria *vocal*, che viene spesso confusa con la classe associata a *bass*.

Le performance rimangono comunque soddisfacenti.



Figure 7: Matrice di confusione normalizzata relativa al caso 2

3.2 Valutazione dei prototipi appresi

Il forward-pass del modello, oltre a restituire i logits utilizzati per la classificazione, produce anche una matrice delle *distanze minime*. Questa matrice ha dimensione pari a $B \times m$, dove B indica il numero di esempi nel batch e m il numero di prototipi. Il valore in cella (i, j) rappresenta la distanza minima tra il prototipo j e tutte le patch dell'output convoluzionale relative all'esempio i . Memorizzando questa matrice per tutti gli esempi del test set, è possibile stimare quanto ciascun prototipo si attivi in media, ovvero quanto frequentemente e con quale intensità esso risulti simile a una regione locale dell'input. Questo consente di analizzare il ruolo semantico dei prototipi e il loro contributo alle decisioni di classificazione del modello.

I risultati sono mostrati nelle figure 9 e 10. Per semplicità vengono mostrati solo i grafici relativi ai primi prototipi associati ad ogni classe. Per ciascun prototipo vengono calcolate due metriche che esprimono la bontà di un prototipo:

- **Ratio score:** rappresenta il rapporto tra la distanza media intra-classe e la distanza media inter-classe. Valori prossimi a 0 indicano una buona capacità discriminativa del prototipo, mentre valori prossimi a 1 suggeriscono una scarsa separazione tra classi. Valori maggiori di 1 indicano invece un prototipo non discriminativo, poiché risulta mediamente più distante dagli esempi della propria classe che da quelli di classi diverse.
- **Separazione:** rappresenta la differenza tra la distanza media inter-classe e la distanza media intra-classe. Valori elevati indicano una migliore capacità del prototipo di distinguere tra classi differenti.

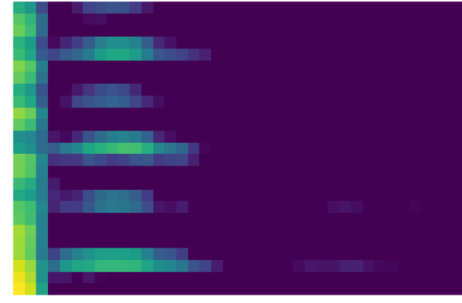


Figure 8: Prototipo 0 specifico della classe *bass*

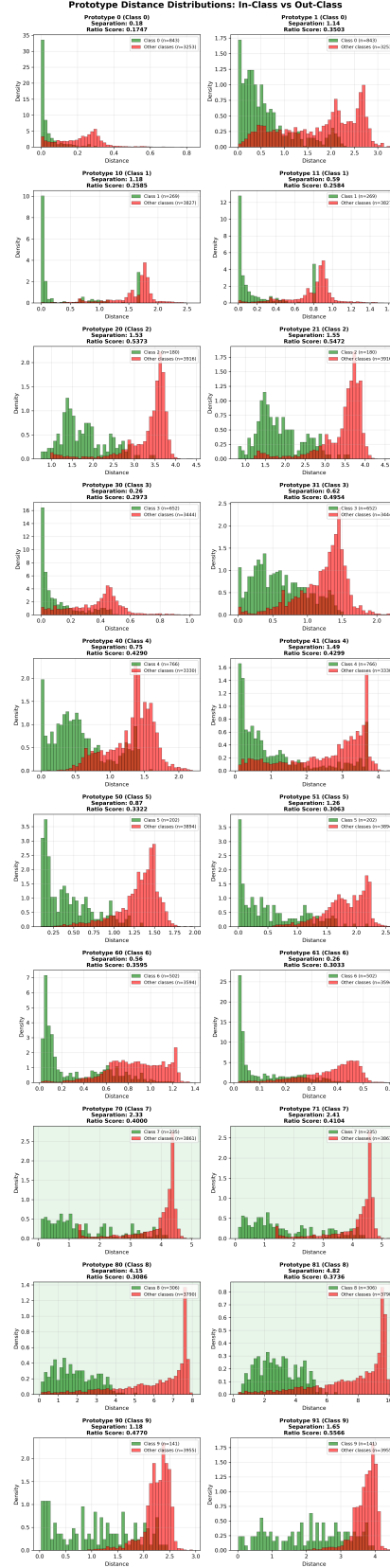


Figure 9: Istogrammi per la valutazione dei prototipi relativa al caso 2 su test set reale

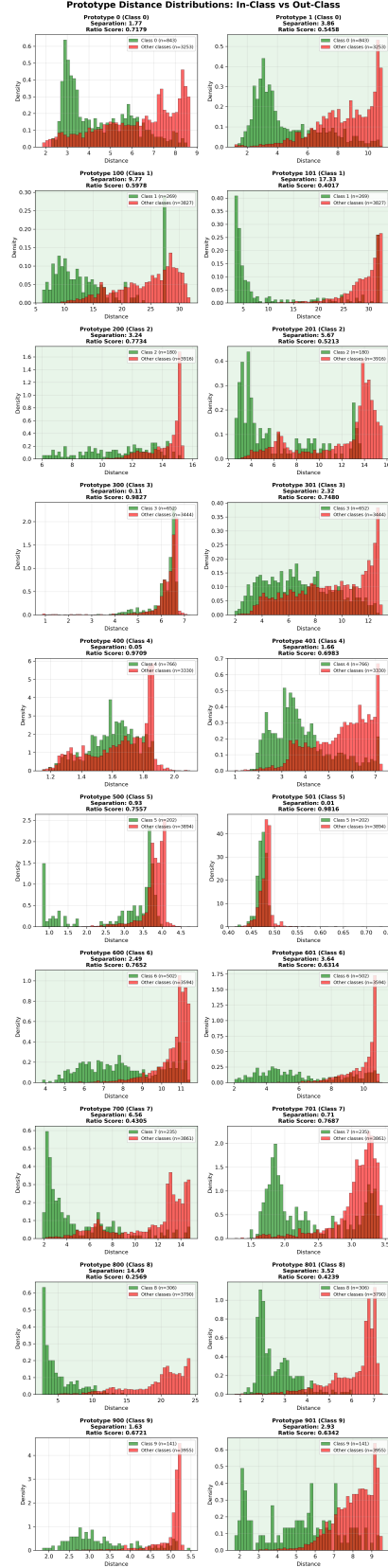


Figure 10: Istogrammi per la valutazione dei prototipi relativa al caso 3 su test set reale

4 Conclusioni

L'applicazione di un modello interpretabile basato sull'apprendimento dei prototipi al task di classificazione sul dataset NSynth ha evidenziato alcune limitazioni. Le performance di generalizzazione ottenute si sono attestate mediamente intorno al 75%, un risultato che suggerisce margini di miglioramento considerevoli. Questa accuratezza contenuta può essere attribuita a diversi fattori tra cui la natura intrinseca del dataset NSynth dato che gli esempi presentano una grande variabilità timbrica e sonora e lo sbilanciamento tra le classi.

Per quanto riguarda, invece, il test set artificiale, le performance sono piuttosto basse indicando che le distanze dai prototipi appresi divergono in modo significativo mostrando una dissimilarità tra i prototipi e gli input.